A Cost-Effective, Automated Approach to Disaster Recovery and Backup for Small Businesses on AWS

Keywords: RTO, RPO, DR, Automation, Backup, Quantitative

Abstract

Chapter 1: Introduction

A very common question from businesses new to the AWS cloud often ask is how does AWS handle disaster recovery and backup? AWS addresses this with a fully integrated suite of managed services, eliminating the need for businesses to build and maintain on-premises DR infrastructure. AWS centralizes backup management, automates replication, and offers flexible DR strategies tailored to unique recovery time objectives (RTO) and recovery point objectives (RPO). The two metrics that define every AWS backup and disaster recovery plan. RTO defines the longest period a workload can remain unavailable while RPO defines the greatest volume of data organisation can afford to lose following a disaster. Cost models of disaster recovery consistently show that the price of a recovery strategy scales closely with how much RTO it removes (Alhazmi and Malaiya, 2013), which raises the question this dissertation investigates: how far RTO and RPO can be improved for a small company without adopting a recovery model priced for a much larger organisation.

1.1 Aims and Objectives

To design, implement and evaluate a cost-effective, AWS-native disaster recovery and backup strategy suited to the operational and financial constraints of small businesses.

The paper is guided by the these key objectives: to review existing literature on cloud-based disaster recovery, identifying the trade-offs between cost, automation, and recovery performance across current AWS DR strategies.

To design and implement a representative small-business workload on AWS, incorporating automated backup, backup protection, monitoring, and Infrastructure-as-Code provisioning.

To detect genuine service failure through layered monitoring built on CloudWatch Synthetics canaries, application error alarms and AWS Health events and to evaluate the system's recovery performance against defined RTO and RPO targets.

To assess whether the proposed solution offers a viable middle ground between manual backup-and-restore and high-cost hot-standby or active-active strategies for resource-constrained organisations.

1.2 Research Questions

This paper is structured around the following research questions:

What combination of AWS-native services can deliver automated, cost-effective disaster recovery for small companies without dedicated infrastructure staff?
How does the proposed architecture perform against defined RTO and RPO targets when recovery is triggered by automated multi-signal failure detection?
To what extent does Infrastructure as Code (Terraform, GitHub Actions) improve the reproducibility and reliability of disaster recovery compared with manually managed recovery processes?
What cost and operational trade-offs distinguish this approach from existing strategies such as hot standby and active-active recovery?

Chapter 2: Literature Review / Related Work

2.1 Comprehensive Overview of Existing Literature

Cloud-Based Disaster Recovery and Recovery Objectives

With the growing adoption of cloud infrastructure by small businesses, there is a need for disaster recovery systems that can restore applications and data after service disruption, system failure, or disaster events. Disaster recovery is a planned and organised system of policies, procedures, and technical mechanisms used to restore disrupted information technology services after failures or disaster events (Sharma, 2026). In cloud computing, this is commonly referred to as cloud-based disaster recovery or Disaster Recovery as a Service, where backup, replication, restore testing, and recovery processes are supported through cloud services instead of traditional on-premises recovery infrastructure. This is important for small companies because the fastest recovery solution is not always the most suitable if it requires expensive standby infrastructure.

AWS Disaster Recovery Strategies

Analysis of cloud disaster recovery identify a spectrum of recovery strategies from backup and restore through pilot light and warm standby to multi-site active-active, often characterised as cold, warm, and hot options (Alhazmi and Malaiya, 2013; Lenk and Tai, 2014). These approaches provide different balances between cost, complexity, RTO, and RPO. Backup and restore is generally the most cost-effective option because duplicate infrastructure does not need to run continuously. However, it can require hours of recovery effort if restoration is manual, untested, or dependent on several administrator actions (Alhazmi and Malaiya, 2013).

Pilot light and warm standby strategies reduce recovery time by keeping some recovery resources available before failure occurs. In a pilot light strategy, core workload components are replicated or prepared while application servers may remain inactive until recovery is required. Warm standby provides a scaled-down but functional version of the production environment. Multi-site active-active provides the fastest recovery because traffic can be served from multiple active locations but this approach is normally the most expensive and complex (Alhazmi and Malaiya, 2013). These options show

the main challenge addressed in this dissertation. improving RTO and RPO without adopting a recovery model that is too expensive for small organisations.

Cost and Performance Trade-Offs in Cloud Disaster Recovery

Several studies highlight the cost-performance tension in cloud disaster recovery. Hot standby disaster recovery continuously replicates applications to a secondary region, allowing faster recovery during failure. Performability is a composite measure that combines the performance and dependability of a system (Andrade and Nogueira, 2019). This is relevant because a disaster recovery solution should not only be judged by whether recovery is possible, but also by how reliably and efficiently the system performs after recovery.

However, hot standby approaches require secondary infrastructure to remain continuously available. This increases operational cost and may not be suitable for small businesses with limited budgets. Cost-performance research also shows that cloud disaster recovery often depends on a secondary part of the IT system, such as a cold spare, hot spare, or partial production environment (Barbierato et al., 2023). While these designs improve recovery capability, they can also increase infrastructure cost, especially when recovery resources remain idle for long periods.

The economic impact of cloud disaster recovery is therefore an important consideration. Cloud-based recovery can reduce dependence on physical standby infrastructure, but organisations must still balance setup cost, storage cost, data transfer cost, and downtime loss. For small companies, the recovery design must provide enough resilience without creating unnecessary ongoing cost.

Quantitative studies reinforce this trade-off. Cost-performance modelling of cloud systems shows that reducing recovery cost by managing infrastructure abstraction in software rather than dedicated hardware can risk application availability performance if not carefully tuned (Naik, 2016). Cost-based evaluations of disaster recovery alternatives confirm that backup-and-restore remains the appropriate strategy for small businesses precisely because pilot light, warm standby, and multi-site active-active options escalate cost roughly in proportion to how much they reduce RTO (Alhazmi and Malaiya, 2013; Wood et al., 2010). This same pattern appears in broader surveys of cloud backup and disaster recovery, which identify cost-performance trade-offs as a recurring constraint on DR design choices rather than a provider-specific artefact (Suguna and Suhasini, 2014). For small businesses specifically, published SMB-focused guidance stresses that resource constraints make off-site, cloud-based backup more practical than traditional standby infrastructure, even though it typically comes with slower recovery than hot standby (Wowrack, 2024).

Automation, Restore Testing, and Backup Validation

Automation is a key requirement in disaster recovery because manual recovery processes can increase RTO and introduce human error. A manual process may require administrators to locate backups, provision infrastructure, restore data, reconfigure services, update networking, and validate the application before users can access it again. In a real failure scenario, these steps can delay recovery and increase operational risk.

Modern cloud backup platforms support scheduled backups, recovery point selection, restore capability and restore testing (Wowrack, 2024). These features are relevant because they allow backup and restore processes to become more repeatable and measurable. A backup strategy that is not tested cannot be assumed to meet its RTO or RPO targets. Restore testing is therefore important for validating whether recovery points can be used during a disaster.

Implementation-focused studies also support the importance of automated recovery. Kubernetes cluster disaster recovery using AWS demonstrates how automation can improve backup and restore processes and allow recovery time to be measured through practical testing (Bhavsar et al., 2023). Similarly automated Kubernetes disaster recovery using LSTM shows that automation can reduce recovery delays and reduce dependence on direct human intervention (Kim, Choi and Jung, 2024). However, both approaches are based on Kubernetes environments which are more complex than the small-company AWS workloads considered in this paper. Their relevance is therefore mainly in demonstrating the value of automation and measured recovery rather than providing the exact architecture for this project.

Cost-Effective Recovery and Idle Infrastructure

For organisations with constrained resources conventional standby systems can be inefficient because backup servers may remain idle for most of their lifetime while still consuming resources and maintenance effort. Automated disaster recovery architecture for educational institution web infrastructure highlights this issue by showing that standby infrastructure can create unnecessary cost when it is rarely used (Vinisha, Sanjay and Sundar, 2025). This is relevant to small businesses because they may need disaster recovery capability but may not be able to justify always-on duplicate infrastructure.

Cloud-based backup and restore can reduce this cost problem because recovery resources do not always need to run continuously. However, basic backup and restore may result in longer recovery time if the process is manual. This creates the need for a middle-ground solution, more automated and reliable than manual backup and restore but less expensive than hot standby, warm standby or active-active recovery.

Infrastructure as Code in Disaster Recovery

A separate strand of literature focuses on Infrastructure as Code (IaC) as a mechanism for making disaster recovery repeatable rather than manually reconstructed. Case-study work applying Terraform and Kubernetes to simulated disaster events found that codifying infrastructure as a deployment pipeline measurably reduced both the number of manual steps and the total time required for recovery (Bahaweres and Najib, 2023). This finding is consistent with systematic reviews of IaC research which reports benefits for repeatability, auditability and provisioning speed across tools such as Terraform (Rahman, Mahdavi-Hezaveh and Williams, 2019) and with work establishing IaC as a core DevOps practice for automated, repeatable environment management in CI/CD settings (Artač et al., 2017). Earlier work proposing IaC-based cloud DR similarly combined strategies such as warm standby, backup and restore and pilot light with Terraform-defined infrastructure to reduce data-loss risk from events

such as ransomware (Lavriv et al., 2018). Taken together, this literature supports treating IaC not as a convenience but as a core component of automated, low-cost disaster recovery, and directly informs the choice to build the proposed system entirely around Terraform and GitHub Actions rather than manual provisioning (Guerriero et al., 2019).

Backup Protection, Monitoring and Security

A disaster recovery strategy is only useful if backups remain protected and recoverable during a failure. Backup vaulting solutions that support governance and compliance-mode locking help to prevent accidental or unauthorised changes to backup vaults (SentinelOne, 2026; Eon, 2026). This is relevant because backups that can be deleted, altered, or misconfigured may not be reliable during a disaster.

Monitoring and auditing are also important parts of disaster recovery management. Services such as Amazon CloudWatch and AWS CloudTrail can be used to monitor backup activity, detect failures, generate alerts, and record recovery-related actions. These controls support operational visibility and help ensure that backup and restore processes are not only automated, but also observable and auditable. For small organisations, this is important because recovery failure may not be caused only by missing backups, but also by a lack of monitoring, poor access control, or untested restoration procedures.

Backup immutability is an increasingly prominent theme in this area of the literature. Because ransomware attacks now routinely target backup repositories in addition to production systems, industry security research argues that write-once-read-many (WORM) mechanisms, such as object lock and compliance-mode vaults, are necessary to guarantee that at least one clean recovery point survives an attack, since deletion or modification is blocked even for administrative accounts during the retention period (SentinelOne, 2026; Eon, 2026). This directly motivates the use of AWS Backup Vault Lock in governance mode within the proposed system, rather than relying on access control alone to protect recovery points.

Research Gap and Proposed System

The reviewed literature shows that existing disaster recovery work addresses several important areas, but often separately. Hot standby and active-active approaches can reduce RTO, but they require additional infrastructure and increase cost (Alhazmi and Malaiya, 2013; Andrade and Nogueira, 2019). Backup and restore is more cost-effective, but recovery may be slow if it is manual, untested, or poorly monitored (Alhazmi and Malaiya, 2013; Wowrack, 2024). Kubernetes-based studies demonstrate the value of automation and recovery testing, but they focus on complex containerised environments rather than simple AWS workloads for small companies (Bhavsar et al., 2023; Kim, Choi and Jung, 2024). Hybrid cloud recovery research highlights the cost issue of idle standby infrastructure, but it is not focused on a fully AWS-native small-company environment (Vinisha, Sanjay and Sundar, 2025).

Therefore, the gap addressed in this paper is the limited practical evaluation of an AWS-native disaster recovery and backup strategy that combines automation, cost-effectiveness, restore testing, backup protection, monitoring, and improved RTO/RPO for small businesses. It focuses on designing and evaluating a practical AWS-based solution that is more reliable than manual backup and restore, but less

expensive and less complex than hot standby, active-active, or Kubernetes-based disaster recovery architectures.

The proposed system will use selected AWS-native services to automate backup scheduling, support restore testing, protect backup integrity, and evaluate recovery performance against defined RTO and RPO targets. The evaluation will consider backup frequency, restore success, recovery time, estimated data loss, automation level, monitoring capability, backup protection, and infrastructure cost.

2.2 Critical Analysis of Existing Studies

The reviewed studies can be compared using parameters that are directly relevant to this paper, including recovery approach, methodology, cost focus, automation, RTO/RPO consideration, and limitations.

Study	Main focus	Methodology / approach	Key limitation	Relevance to this dissertation
Sharma (2026)	Cloud-based disaster recovery for Workday applications	Analytical modelling and simulation	Workday-specific and enterprise-focused	Supports the disaster recovery definition and the importance of RTO/RPO
Andrade and Nogueira (2019)	Hot standby cloud disaster recovery	Performability evaluation	Expensive due to secondary infrastructure	Shows strong recovery performance but high cost
Barbierato et al. (2023)	Cost and performance of cloud disaster recovery	Cost-performance evaluation	Not focused on a small AWS implementation	Supports the cost-performance trade-off
Bhavsar et al. (2023)	Kubernetes disaster recovery using AWS	Implementation and testing	Kubernetes can be costly and complex for small organisations	Supports automation and restore testing
Kim, Choi and Jung (2024)	Automated Kubernetes disaster recovery using LSTM	Automated recovery system	Too complex for simple small-company workloads	Supports automation but also shows the risk of complexity
Vinisha, Sanjay and Sundar (2025)	Automated disaster recovery for educational web infrastructure	Hybrid cloud implementation	Hybrid and education-specific	Supports cost-effective automation and the avoidance of idle standby resources
Naik (2016)	Cost-performance trade-offs in cloud systems	PhD thesis; adaptive systems modelling	General cloud systems, not DR-specific	Supports the cost-performance framing used throughout this dissertation
Alhazmi and Malaiya (2013)	Cost-based evaluation of disaster recovery plan alternatives	Analytical cost modelling	Analytical model, no cloud implementation	Directly supports the strategy and cost framing for small companies

Study	Main focus	Methodology / approach	Key limitation	Relevance to this dissertation
Suguna and Suhasini (2014)	Survey of cloud data backup and disaster recovery techniques resilience	Survey	Descriptive survey, no implementation	Broadens the cost trade-off argument beyond a single provider
Bahaweres and Najib (2023)	IaC impact on DR speed and complexity	Case study, simulated disaster event	Uses Kubernetes rather than a simple AWS workload	Provides direct evidence that IaC reduces recovery steps and time
Rahman, Mahdavi-Hezaveh and Williams (2019)	Systematic mapping of Infrastructure as Code research	Comparative analysis	Broad mapping study, not DR-specific	Justifies the choice of Terraform for reproducible provisioning
Lavriv et al. (2018)	Combining backup strategies with Infrastructure as Code	Conceptual design and prototype	Early-stage, OpenStack-based rather than AWS	Early precedent for combining IaC with backup strategy selection
Eon (2026)	Backup immutability and governance-mode locking	Industry analysis	Vendor-authored, not independently tested	Supports the Vault Lock governance-mode design choice
Wowrack (2024)	DR strategy guidance for small and medium businesses	Practitioner guidance	Descriptive, not AWS-specific or tested	Supports the SME cost-constraint framing central to this dissertation

The comparison shows that the existing literature provides useful foundations but does not fully address the exact focus of this paper. Performance-focused work demonstrates that low RTO can be achieved through standby infrastructure, but this increases cost. Cost-focused work explains the trade-off between recovery capability and infrastructure cost, but it does not provide a small-company AWS implementation. Automation-focused studies show that backup and restore can be improved, but they are often based on Kubernetes or hybrid cloud environments (Bhavsar et al., 2023; Kim, Choi and Jung, 2024; Bahaweres and Najib, 2023). Cost models of cloud recovery alternatives confirm that strategy cost scales closely with the reduction in RTO achieved, but stop at analytical comparison rather than a working implementation (Alhazmi and Malaiya, 2013). Infrastructure-as-Code studies show that Terraform-based provisioning can reduce recovery steps and time, but are usually demonstrated on enterprise or Kubernetes workloads rather than a simple small-company AWS stack (Bahaweres and Najib, 2023; Rahman, Mahdavi-Hezaveh and Williams, 2019; Lavriv et al., 2018). Industry research on backup immutability supports governance-mode protection as a defence against ransomware, but does not evaluate this within an end-to-end, measured recovery pipeline (SentinelOne, 2026; Eon, 2026). None of the reviewed literature evaluates a complete small-company AWS implementation with measured RTO, RPO, restore success, automation level, and cost, which is the gap this dissertation addresses.

2.3 Summary

This chapter reviewed literature on cloud-based disaster recovery, AWS disaster recovery strategies, cost-performance trade-offs, automation, restore testing, and backup protection. The literature shows that disaster recovery must balance cost, complexity, RTO, and RPO. Expensive strategies such as hot standby and active-active can improve recovery time, but they may not be suitable for small businesses. Backup and restore is more cost-effective, but it can result in longer recovery time if it is manual or untested.

The gap identified from the literature is the need for a practical AWS-native backup and disaster recovery solution that combines automation, cost-effectiveness, restore testing, backup protection, monitoring, and improved RTO/RPO for small businesses. This dissertation addresses that gap by implementing and evaluating selected AWS services against practical recovery and cost criteria.

Chapter 3: Methodology

3.1 System Overview

This paper takes an experimental, implementation-based approach, using AWS-native services to design, build and evaluate a disaster recovery solution aimed specifically at small businesses. The decision not to rely on surveys or purely theoretical analysis comes directly from the gap identified in Chapter 2. Most of the existing literature treats disaster recovery either conceptually or within complex environments such as Kubernetes or hybrid cloud setups (Bhavsar et al., 2023; Kim, Choi and Jung, 2024; Bahaweres and Najib, 2023). What is missing is a practical evaluation of a simple, AWS-native solution that a small organisation with limited budget and technical staff could realistically adopt.

The research is quantitative in nature. A representative small-company workload is deployed on AWS and managed entirely through Infrastructure as Code, with a backup and recovery architecture built around it. Rather than staging failures manually the architecture is evaluated against genuine failure signals. A CloudWatch Synthetics canary, load balancer error alarms and AWS Health events monitor the deployment and recovery is triggered automatically once a composite alarm confirms a sustained failure. This produces measurable results across RTO, RPO, restore success rate, automation level and cost rather than descriptive comparison alone, as was largely the case in the reviewed literature.

The overall design sits deliberately between the two extremes discussed in Chapter 2. On one side is manual backup and restore, which is cheap but slow and prone to human error. On the other is hot standby or active-active recovery, which is fast but expensive and often more than a small business needs. This project tries to find a workable middle ground: automated enough to reduce delay and risk, but without the ongoing cost of infrastructure that sits idle most of the time. All components are provisioned and managed through Terraform, with GitHub Actions handling the automation side — provisioning, updates, and recovery workflows — so that the whole setup reflects something a small team could genuinely maintain, rather than a one-off academic exercise.

3.2 System Components

To keep the implementation grounded rather than abstract, this paper is built around a fictional small e-commerce vendor called CraftHaven, which sells handmade home and lifestyle products through its own online store. CraftHaven is a reasonable stand-in for the kind of organisation this research is aimed at: a small team of under ten people, no dedicated infrastructure engineer, no existing disaster recovery plan, and a business that is entirely dependent on its website and order data to function. If the site goes down or order data is lost, there's no fallback which is exactly the kind of risk this dissertation is trying to address.

The system is made up of the following components:

Application layer

CraftHaven's web application runs on an Amazon EC2 instance behind an Application Load Balancer, handling the storefront and order processing. Customer records and order history are stored in an Amazon RDS (MySQL) database, while product images and other static assets sit in Amazon S3. Everything runs inside an Amazon VPC, with the web tier and database tier separated across public and private subnets.

Backup layer

AWS Backup handles scheduled backups across EC2/EBS volumes, the RDS instance, and the S3 bucket, using defined backup plans and retention rules, with recovery points stored centrally in a managed backup vault.

Protection layer

AWS Backup Vault Lock is configured in governance mode so that backups can't be accidentally or maliciously deleted or altered, a safeguard that's easy to overlook but matters a great deal in practice.

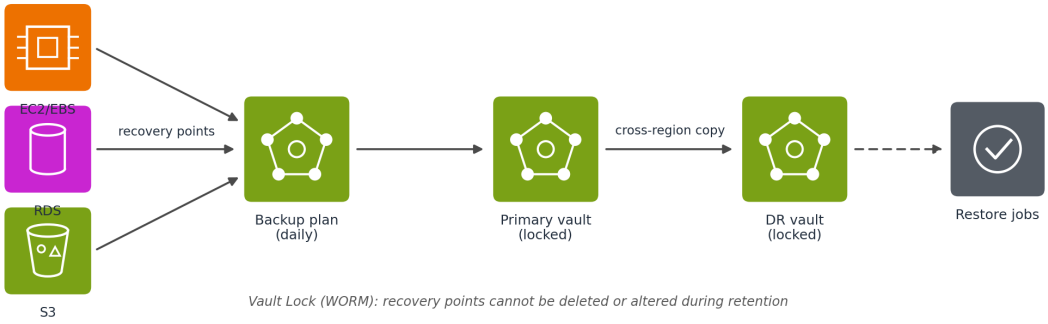

Figure 1. Backup and protection flow.

Recovery layer

Rather than relying on someone manually triggering restores and rebuilding infrastructure step by step, automated restore scripts handle most of that process. This is really the core idea behind the whole project: cutting down the manual work involved in backup-and-restore without going as far as running duplicate infrastructure all the time.

Failure detection and failover layer

Failure detection uses three independent signals rather than a single check. A CloudWatch Synthetics canary runs on a fixed schedule and executes a scripted browse and checkout journey against the live site. This works even when no customers are browsing, which matters for a small business whose traffic can drop to zero overnight. CloudWatch alarms on the load balancer metrics watch for elevated HTTP 5XX error rates and unhealthy targets and catch application failures under real load. AWS Health events delivered through EventBridge report service or region level incidents that application metrics cannot explain on their own. The canary and error alarms feed a CloudWatch composite alarm so a single blip cannot trigger recovery. When the composite alarm fires an EventBridge rule starts the restore

workflow. Route 53 remains the public DNS entry point and once a restore completes the workflow updates the DNS record to point the domain at the recovered environment.

Monitoring layer

Amazon CloudWatch tracks backup job status and application health and raises alerts when something fails. AWS CloudTrail logs backup, restore, and configuration changes, so there's a clear audit trail if something needs to be investigated later.

Infrastructure as Code and automation layer

The entire environment is defined in Terraform and version-controlled in a GitHub repository. GitHub Actions drives the automation: applying Terraform changes when code is merged, running scheduled restore tests, and executing the restore workflow automatically when a genuine failure is detected.

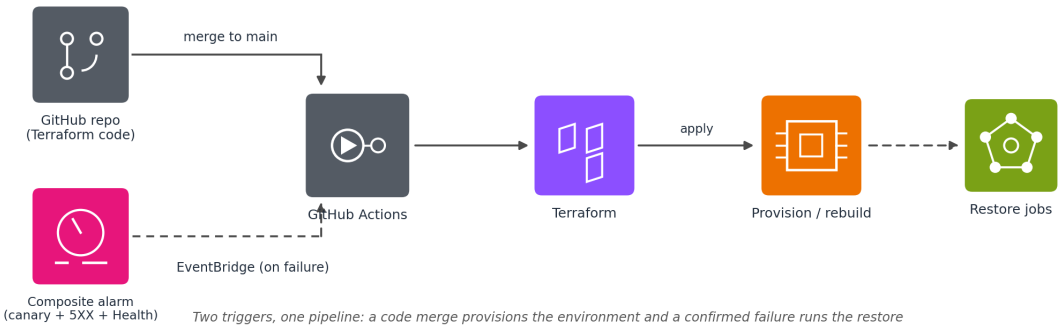

Figure 2. Infrastructure as Code pipeline.

The advantage of this setup is that it's not just the data that can be recovered — the infrastructure itself is disposable and rebuildable, which matters a lot for a small company that can't afford to have someone rebuild servers by hand under pressure.

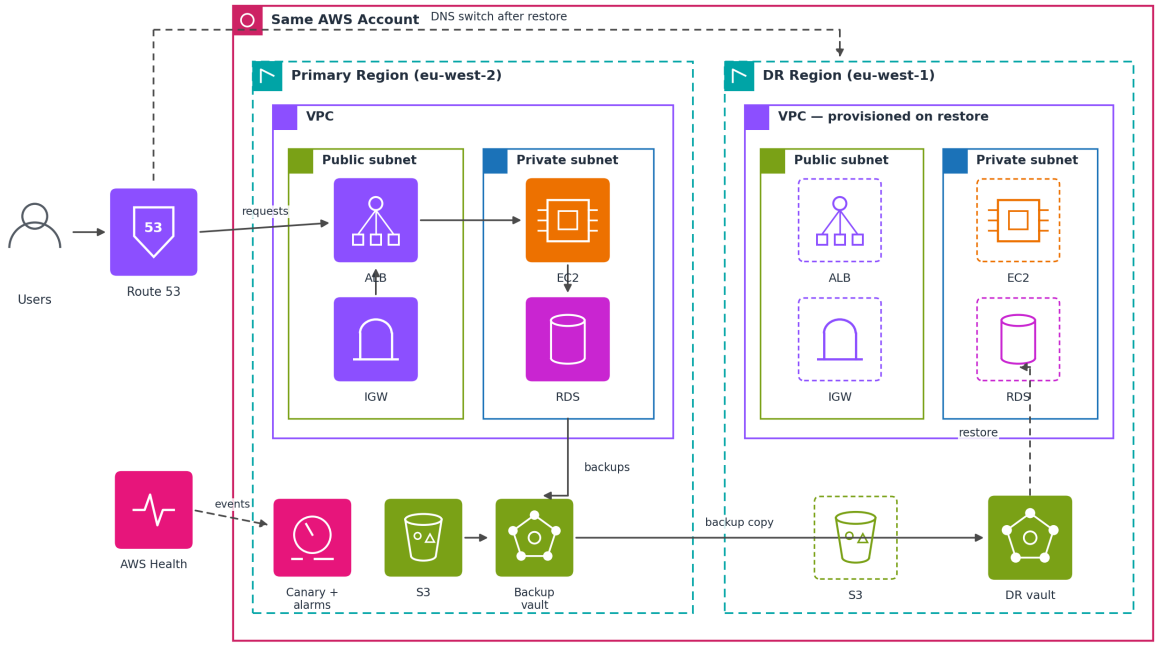

Figure 3. Full Network architecture.

3.3 Architecture Design

The architecture follows a straightforward flow, with Terraform and GitHub Actions acting as the glue between provisioning, backup, and recovery.

Provisioning. Terraform configuration, stored in GitHub, defines the entire CraftHaven environment. When changes are merged to the main branch, a GitHub Actions workflow runs terraform plan and terraform apply, provisioning everything from the VPC down to the CloudWatch alarms.

Normal operation. Once running, Route 53 resolves customer requests to the load balancer in the primary region (eu-west-2). The EC2-hosted application serves the traffic, writes to RDS and pulls product images from S3. The Synthetics canary probes the site on its schedule throughout.

Automated backup. AWS Backup runs on a set schedule (daily, in this case) creating recovery points for EC2/EBS, RDS and S3 all stored in the locked backup vault with each recovery point automatically copied to a second locked vault in the DR region (eu-west-1) so that restores never depend on the health of the primary region.

Monitoring. CloudWatch keeps an eye on backup jobs and application health throughout, while CloudTrail quietly logs every backup, restore, and infrastructure change in the background.

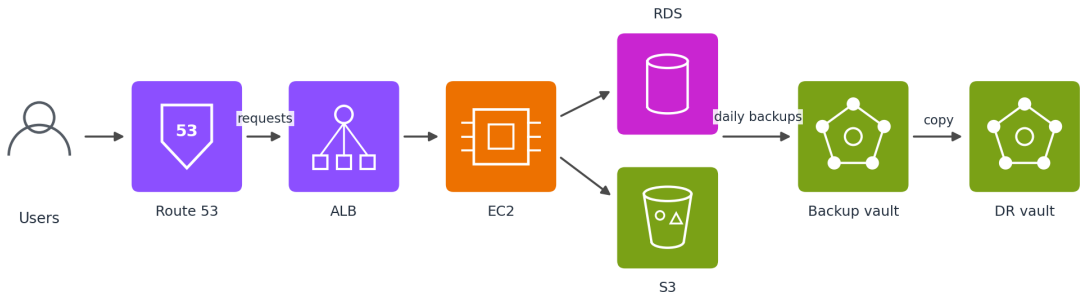

Figure 4. Steady-state operation with daily backups replicating cross-region.

Failure detection. The system does not wait for customers to hit a broken site. A CloudWatch Synthetics canary probes the storefront every few minutes and fails its run if the page does not load or the scripted checkout flow breaks. In parallel CloudWatch alarms watch the load balancer for elevated 5XX error rates and unhealthy targets. AWS Health events arrive through EventBridge when AWS itself reports a problem affecting EC2, RDS or the region. The canary alarm and the error rate alarm are combined in a composite alarm that must stay in breach across several evaluation periods before a disaster is declared. This filters out transient blips while still detecting a real outage within minutes even at zero traffic.

Recovery. When the composite alarm fires an EventBridge rule calls the GitHub Actions restore workflow through an API destination. No human has to notice the outage first. AWS Backup restore jobs rebuild the data in the DR region (eu-west-1) from the replicated recovery points while Terraform provisions the surrounding infrastructure from the same code that built the primary region.

Failover. Once the restore workflow's smoke checks pass and a canary run against the DR endpoint succeeds, the workflow updates the Route 53 record so the domain points at the recovered environment. From the customer's point of view nothing changes. The same domain simply starts resolving to the DR region. Recovery is judged by when customers can use the site again, not by when an engineer declares the system restored.

Validation. Once traffic is flowing to the restored environment, the application is checked for availability and data integrity. RTO is measured from the moment the composite alarm fired to the moment customers were redirected to a working site and RPO from the last replicated recovery point, with results recorded against the criteria set out in Section 3.4.

Figure 5. Detection and recovery: a confirmed composite alarm triggers automated restore, ending with the DNS switch.

3.4 Evaluation Criteria

The solution is evaluated against a set of criteria that come directly out of the trade-offs discussed in Chapter 2:

Criterion	Definition	Measurement Method
Recovery Time Objective (RTO)	Time from failure trigger to full application availability	Timed from composite alarm firing to customers redirected to restored service
Recovery Point Objective (RPO)	Data loss window between last successful backup and point of failure	Calculated from backup schedule and failure timestamp
Restore Success Rate	Proportion of restore attempts completing without manual intervention	Measured across repeated trials
Automation Level	How much of backup/restore is manual vs. automated	Step count tracked via GitHub Actions workflow logs
Backup Protection	Whether backups are safe from unauthorised deletion or change	Verified through Vault Lock configuration testing
Monitoring and Auditability	Whether backup/restore actions are visible and logged	Verified via CloudWatch alerts and CloudTrail logs
Cost	Estimated AWS cost of running the solution	Calculated via AWS Cost Explorer / pricing calculator
Infrastructure Reproducibility	Whether the environment can be rebuilt from code alone	Verified by destroying and re-provisioning via Terraform

Taken together, these criteria let the solution be judged not just on whether recovery is possible, but on whether it strikes a reasonable balance between speed, reliability, automation, reproducibility, and cost — which is really the central question running through the whole literature review.

3.5 Advantages of the Proposed Solution

A few things stand out as genuine advantages of this approach over the strategies covered in Chapter 2.

It's considerably cheaper to run than hot standby or active-active recovery, since there's no duplicate infrastructure sitting idle waiting for a disaster that may never come — while still being faster and more reliable than a purely manual backup-and-restore process. Automating both the backup schedule and the infrastructure provisioning also cuts down heavily on the manual steps and human error that the

literature repeatedly flags as a weak point in traditional backup-and-restore setups (Naik, 2016; Alhazmi and Malaiya, 2013).

Because the whole environment is defined in code, CraftHaven's infrastructure becomes something that can itself be rebuilt after a disaster, not just the data sitting on top of it — which is a step beyond most of the approaches discussed earlier, where the assumption tends to be that the underlying infrastructure is already there (Bahaweres and Najib, 2023; Rahman, Mahdavi-Hezaveh and Williams, 2019). Pairing CloudWatch and CloudTrail into the setup also means backup and recovery actions are visible, addressing a point raised in Chapter 2 that recovery failures are often caused less by missing backups and more by nobody noticing something had gone wrong in the first place. Vault Lock adds another layer on top of that, protecting backups from being deleted or tampered with, which closes a gap that recoverability alone doesn't solve.

Perhaps most importantly for the audience this research is aimed at, none of this requires a dedicated infrastructure engineer on staff. Routine provisioning, backups and recovery all run through pre-built automation pipelines, which fits well with the reality of small businesses that simply don't have the resources to hire specialised staff for this kind of work (Wowrack, 2024).

Chapter 4: Result Analysis

Chapter 5: Conclusion and Future Work

References

Alhazmi, O.H. and Malaiya, Y.K. (2013) 'Evaluating disaster recovery plans using the cloud', 2013 Proceedings Annual Reliability and Maintainability Symposium (RAMS), Orlando, FL, pp. 1–6.

Andrade, E. and Nogueira, B. (2019) 'Performability evaluation of a cloud-based disaster recovery solution for IT environments', Journal of Grid Computing, 17, pp. 603–621.

Artač, M., Borovšak, T., Di Nitto, E., Guerriero, M. and Tamburri, D.A. (2017) 'DevOps: introducing infrastructure-as-code', 2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C), pp. 497–498.

Bahaweres, R.B. and Najib, F.M. (2023) 'Provisioning of disaster recovery with Terraform and Kubernetes: a case study on software defect prediction', 2023 10th International Conference on Electrical Engineering, Computer Science and Informatics (EECSI), pp. 183–189.

Barbierato, E., Iacono, M., Gribaudo, M. and Mastroianni, M. (2023) 'Cost- and performance-based evaluation of cloud-based disaster recovery', Proceedings of the 37th ECMS International Conference on Modelling and Simulation, pp. 568–574.

Bhavsar, S., Agrawal, A., Ropalkar, T., Kamdi, P., Hajare, A., Deshpande, S., Rathi, R. and Garg, D. (2023) 'Kubernetes cluster disaster recovery using AWS', 2023 7th International Conference on Computing, Communication, Control and Automation (ICCUBEA).

Eon (2026) Immutable backups: how they work and best practices. Available at: eon.io (Accessed: 18 July 2026).

Guerriero, M., Garriga, M., Tamburri, D.A. and Palomba, F. (2019) 'Adoption, support, and challenges of infrastructure-as-code: insights from industry', 2019 IEEE International Conference on Software Maintenance and Evolution (ICSME), pp. 580–589.

Kim, J.-B., Choi, J.-B. and Jung, E.-S. (2024) 'Design and implementation of an automated disaster-recovery system for a Kubernetes cluster using LSTM', Applied Sciences, 14(9), 3914.

Lavriv, O. et al. (2018) 'Method of cloud system disaster recovery based on “Infrastructure as a code” concept', 2018 14th International Conference on Advanced Trends in Radioelectronics, Telecommunications and Computer Engineering (TCSET).

Lenk, A. and Tai, S. (2014) 'Cloud standby: disaster recovery of distributed systems in the cloud', Service-Oriented and Cloud Computing (ESOCC 2014), Lecture Notes in Computer Science, 8745, pp. 32–46.

Naik, N. (2016) Exploiting cost-performance tradeoffs for modern cloud systems. PhD thesis. University of Illinois at Urbana-Champaign.

Rahman, A., Mahdavi-Hezaveh, R. and Williams, L. (2019) 'A systematic mapping study of infrastructure as code research', Information and Software Technology, 108, pp. 65–77.

SentinelOne (2026) What are immutable backups? Autonomous ransomware protection. Available at: sentinelone.com (Accessed: 18 July 2026).

Sharma, M. (2026) 'Performance analysis of cloud-based disaster recovery for Workday applications', TechRxiv [Preprint].

Suguna, S. and Suhasini, A. (2014) 'Overview of data backup and disaster recovery in cloud', 2014 International Conference on Information Communication and Embedded Systems (ICICES), Chennai, pp. 1–7.

Vinisha, J., Sanjay, R. and Sundar, D. (2025) 'Automated disaster recovery architecture for educational institutions web infrastructure using hybrid cloud', 2025 2nd International Conference on Artificial Intelligence and Knowledge Discovery in Concurrent Engineering.

Wood, T., Cecchet, E., Ramakrishnan, K.K., Shenoy, P., van der Merwe, J. and Venkataramani, A. (2010) 'Disaster recovery as a cloud service: economic benefits and deployment challenges', HotCloud'10: 2nd USENIX Workshop on Hot Topics in Cloud Computing.

Wowrack (2024) IT disaster recovery strategies for small and medium-sized businesses (SMBs). Available at: wowrack.com (Accessed: 18 July 2026).